第 4 章 RKNN-Toolkit2API 介绍

4.1 RKNN 初始化及对象释放

在使用 RKNNToolkit2 的所有 API 接口时，都需要先调用 RKNN()方法初始化 RKNN 对象，当不再使用该对象时，通过调用该对象的 release()方法进行释放。

初始化 RKNN 对象时，可以设置 verbose 和 verbose_file 参数，从而打印详细的日志信息。其中 verbose 参数指定是否要在终端打印详细日志信息；如果设置了 verbose_file 参数，且 verbose 参数值为 True，日志信息还将写到该参数指定的文件中。

举例如下：

```
#将详细的日志信息输出到终端，并写到 mobilenet_build.log 文件中
    rknn=RKNN(verbose=True,verbose_file='./mobilenet_build.log')
    rknn=RKNN(verbose=True)   #将详细的日志信息输出到终端
    …
    rknn.release()
```

4.2 RKNN 模型配置

在构建 RKNN 模型之前，需要先对模型进行通道均值、量化图片 RGB2BGR 转换、量化类型等配置，这些操作需要通过 config 接口进行配置。

API	config
描述	设置要构建的 RKNN 模型属性。
参数	mean_values：输入的均值。参数格式是一个列表，列表中包含一个或多个均值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如[[128,128,128]]，表示一个输入的三个通道的值减去 128。如果 quant_img_RGB2BGR 设置成 True，则优先做 RGB2BGR 转换，再做减均值。
	std_values：输入的归一化值。参数格式是一个列表，列表中包含一个或多个归一化，值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如[[128,128,128]]，表示设置一个输入的三个通道的值减去均值后再除以 128。如果 quant_img_RGB2BGR 设置成 True，则优先做 RGB2BGR 转换，再减均值和除以归一化值。
	quant_img_RGB2BGR：表示在加载量化图像时是否需要先做 RGB2BGR 的操作。默认值为 False，如果有多个输入，则用列表包含起来，如[True,True,False]。该配置一般用在 Caffe 的模型上，Caffe 的模型训练时大多会对先数据集图像进行 RGB2BGR 转换，此时需要将该配置设为 True。 另外，该配置只对量化图像格式为 jpg/jpeg/png/bmp 有效，npy 格式读取时会忽略

	该配置，因此当模型输入为 BGR 时，npy 也需要为 BGR 格式。 注意：该配置仅用于在量化阶段（build 接口）读取量化图像或量化精度分析（accuracy_analysis 接口），并不会记录在最终的 RKNN 模型中，因此如果模型的输入为 BGR，则在调用 Python-API 的 inference 或 C-API 的 run 函数之前，需要保证传入的图像数据也为 BGR 格式。
	quantized_dtype 表示量化类型，目前支持的量化类型有 asymmetric_quantized-8、asymmetric_quantized-16，默认值为 asymmetric_quantized-8。（asymmetric_quantized-16 目前版本暂不支持）
参数	quantized_algorithm 表示计算每一层的量化参数时采用的量化算法，目前支持的量化算法有：norma、mmse 及 kl_divergence，默认采用 normal 量化算法。 normal 量化算法的特点是速度较快，推荐量化数据量一般为 20-100 张左右，更多的数据量下精度未必会有进一步提升。 mmse 量化算法由于采用暴力迭代的方式，速度较慢，但通常会比 normal 具有更高的精度，推荐量化数据量一般为 20-50 张左右，用户也可以根据量化时间长短对量化数据量进行适当增减。 kl_divergence 量化算法耗时比 normal 多一些，但比 mmse 少，在某些场景下（feature 分布不均匀时）可以得到较好的改善效果。
	quantized_method 表示模型量化的方法，目前支持 layer 或者 channel。 layer 表示每层的 weight 只有一套量化参数； channel 表示每层的 weight 的每个通道都有一套量化参数； 通常情况下 channel 会比 layer 精度更高，默认值为 channel。
	float_dtype:浮点的数据类型，目前支持的数据类型有 float16，默认值为 float16。
	optimization_level：模型优化等级。通过修改模型优化等级，可以关掉部分或全部模型转换过程中使用到的优化规则。该参数的默认值为 3，打开所有优化选项。值为 2 或 1 时关闭一部分可能会对部分模型精度产生影响的优化选项，值为 0 时关闭所有优化选项。
	target_platform：指定 RKNN 模型是基于哪个目标芯片平台生成的。目前支持“rk3566”、“rk3568”、“rk3588”、“rv1103”、“rv1106”。该参数的值大小写不敏感。
	custom_stirng：添加自定义字符串信息到 RKNN 模型，可以在 C API 中通过 query 接口查询到该信息。
	remove_weight：去除 conv2d 权重以生成一个 RKNN 的从模型，该从模型可以与带完整权重的 RKNN 模型共享权重以减少内存消耗。默认值为 False。
	compress_weight：压缩模型权重，可以减小 RKNN 模型的大小。默认值为 False。
	single_core_mode（仅对 3588 有效）：是否仅生成单核模型，可以减小 RKNN 模型的大小和内存消耗。默认值为 False。

举例如下：

```
rknn.config(mean_values=[123.675,116.28,103.53],std_values=[58.395,58.395,58.395])
```

4.3 模型加载

RKNN-Toolkit2 目前支持 Caffe、TensorFlow、TensorFlowLite、ONNX、DarkNet、PyTorch 等模型的加载转换，这些模型在加载时需调用对应的接口，以下为这些接口的详细说明。

4.3.1 Caffe 模型加载接口

API	load_caffe
描述	加载 caffe 模型。
参数	model：caffe 模型文件（.prototxt 后缀文件）所在路径。
	blobs：caffe 模型的二进制数据文件（.caffemodel 后缀文件）所在路径。该参数值可以为 None，RKNNToolkit2 将随机生成权重等参数。
	input_name：caffe 模型存在多输入时，可以通过该参数指定输入层名的顺序，形如['input1','input2','input3']，注意名字需要与模型输入名一致；也可不设定，按 caffe 模型文件（.prototxt 后缀文件）自动给定。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
1    #从当前路径加载 mobilenet_v2 模型
2  ret=rknn.load_caffe(model='./mobilenet_v2.prototxt',blobs='./mobilenet_v2.caffemodel')
```

4.3.2 TensorFlow 模型加载接口

API	load_tensorflow
描述	加载 TensorFlow 模型。
参数	tf_pb：TensorFlow 模型文件（.pb 后缀）所在路径。
	inputs：模型的输入节点（操作数名），支持多个输入节点。所有输入节点名放在一个列表中。
	input_size_list：每个输入节点对应的图片的尺寸和通道数。如示例中的 mobilenet-v1 模型，其输入节点对应的输入尺寸是[[1,224,224,3]]。
	outputs：模型的输出节点（操作数名），支持多个输出节点。所有输出节点名放在一个列表中。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
#从当前目录加载 ssd_mobilenet_v1_coco_2017_11_17 模型
    ret=rknn.load_tensorflow(
     tf_pb='./ssd_mobilenet_v1_coco_2017_11_17.pb',inputs=['Preprocessor/sub'],outputs=['concat','concat_1'
       ],input_size_list=[[1,300,300,3]])
```

4.3.3 TensorFlowLite 模型加载接口

API	load_tflite
描述	加载 TensorFlowLite 模型。
参数	model：TensorFlowLite 模型文件（.tflite 后缀）所在路径。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
    #从当前目录加载 mobilenet_v1 模型
    ret=rknn.load_tflite(model='./mobilenet_v1.tflite')
```

4.3.4 ONNX 模型加载

API	load_onnx
描述	加载 ONNX 模型。
参数	model：ONNX 模型文件（.onnx 后缀）所在路径。
	inputs：模型输入节点（操作数名），支持多个输入节点。所有输入节点名放在一个列表中。
	input_size_list：每个输入节点对应的图片的尺寸和通道数。如示例中的 mobilenet-v1 模型，其输入节点对应的输入尺寸是[[1,224,224,3]]。
	outputs：模型的输出节点（操作数名），支持多个输出节点。所有输出节点名放在一个列表中。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
    #从当前目录加载 yolov5 模型
    ret=rknn.load_onnx(model='./yolov.onnx')
```

4.4.5 DarkNet 模型加载接口

API	load_darknet

描述	加载 DarkNet 模型。
参数	model：DarkNet 模型文件（.cfg 后缀）所在路径。
	weight：权重文件（.weights 后缀）所在路径。
返回值	0：导入成功 -1：导入失败

举例如下：

```
#从当前目录加载 yolov3-tiny 模型
ret=rknn.load_darknet(model='./yolov3-tiny.cfg',weight='./yolov3.weights')
```

4.4.6 PyTorch 模型加载接口

API	load_pytorch
描述	加载 PyTorch 模型。 支持量化感知训练（QAT）模型，但需要将 torch 版本更新至 1.9.0 以上。
参数	model：PyTorch 模型文件（.pt 后缀）所在路径，而且需要是 torchscript 格式的模型。必填参数。
	input_size_list：每个输入节点对应的图片的尺寸和通道数。例如[[1,1,224,224],[1,3,224,224]]表示有两个输入，其中一个输入的 shape 是[1,1,224,224]，另外一个输入的 shape 是[1,3,224,224]。必填参数。
返回值	0：导入成功
	-1：导入失败

举例如下：

```
#从当前目录加载 resnet18 模型
ret=rknn.load_pytorch(model='./resnet18.pt',input_size_list=[[1,3,224,224]])
```

4.4 构建 RKNN 模型

API	build
描述	依照加载的模型结构及权重数据，构建对应的 RKNN 模型。
	do_quantization：是否对模型进行量化，值为 True 或 False。

参数	dataset：量化校正数据的数据集。目前支持文本文件格式，用户可以把用于校正的图片（jpg 或 png 格式）或 npy 文件路径放到一个.txt 文件中。文本文件里每一行一条路径信息。如： a.jpg b.jpg 或 a.npy b.npy 如有多个输入，则每个输入对应的文件用空格隔开，如： a.jpg a2.jpg b.jpg b2.jpg 或 a.npy a2.npy b.npy b2.npy 注：量化图片建议选择与预测场景较吻合的图片。 当 npy 文件维度为 4 时，需要将 layout 转为 nhwc。（其余维度不需做此转换）
	rknn_batch_size：模型的输入 Batch 参数调整，默认值为 1。如果大于 1，则可以在一次推理中同时推理多帧输入图像或输入数据，如 MobileNet 模型的原始 input 维度为[1,224,224,3]，output 维度为[1,1001]，当 rknn_batch_size 设为 4 时，input 的维度变为[4,224,224,3]，output 维度变为[4,1001]。注： 1. rknn_batch_size 的调整并不会提高一般模型在 NPU 上的执行性能，但却会显著增加内存消耗以及增加单帧的延迟。 2. rknn_batch_size 的调整可以降低超小模型在 CPU 上的消耗，提高超小模型的平均帧率。（适用于模型太小，CPU 的开销大于 NPU 的开销）。 3. rknn_batch_size 的值建议小于 32，避免内存占用太大而导致推理失败。 4. rknn_batch_size 修改后，模型的 input/output 维度会被修改，使用 inference 推理模型时需要设置相应的 input 的大小，后处理时，也需要对返回的 outputs 进行处理。
返回值	0：构建成功
	-1：构建失败

举例如下：

```
#构建 RKNN 模型，并且做量化
ret=rknn.build(do_quantization=True,dataset='./dataset.txt')
```

4.5 导出 RKNN 模型

通过本工具构建的 RKNN 模型通过该接口可以导出存储为 RKNN 模型文件，用于模型部署。

API	export_rknn
描述	将 RKNN 模型保存到指定文件中（.rknn 后缀）。
参数	export_path：导出模型文件的路径。
返回值	0：导出成功
	-1：导出失败

举例如下：

```
……
#将构建好的 RKNN 模型保存到当前路径的 mobilenet_v1.rknn 文件中
ret=rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

4.6 加载 RKNN 模型

API	load_rknn
描述	加载 RKNN 模型。加载后的模型仅限于连接 NPU 硬件进行推理或获取性能数据等。不能用于模拟器或精度分析等。
参数	path：RKNN 模型文件路径。
返回值	0：加载成功
	-1：加载失败

举例如下：

```
#从当前路径加载 mobilenet_v1.rknn 模型
ret=rknn.load_rknn(path='./resnet18.rknn')
```

4.7 初始化运行时环境

在模型推理或性能评估之前，必须先初始化运行时环境，明确模型的运行平台（具体的目标硬件平台或软件模拟器）。

API	init_runtime
描述	初始化运行时环境。确定模型运行的设备信息（硬件平台信息、设备 ID）；性能评估时是否启用 debug 模式，以获取更详细的性能信息。

参数	target：目标硬件平台，支持“rk3566”、“rk3568”、“rk3588”、“rv1103”、“rv1106”。默认为 None，即在 PC 使用工具时，模型在模拟器上运行。 注：target 设为 None 时，需要先调用 build 或 hybrid_quantization 接口才可让模型在模拟器上运行。
	device_id：设备编号，如果 PC 连接多台设备时，需要指定该参数，设备编号可以通过“list_devices”接口查看。默认值为 None。
	perf_debug：进行性能评估时是否开启 debug 模式。在 debug 模式下，可以获取到每一层的运行时间，否则只能获取模型运行的总时间。默认值为 False。
	eval_mem：是否进入内存评估模式。进入内存评估模式后，可以调用 eval_memory 接口获取模型运行时的内存使用情况。默认值为 False。
	async_mode：是否使用异步模式。调用推理接口时，涉及设置输入图片、模型推理、获取推理结果三个阶段。如果开启了异步模式，设置当前帧的输入将与推理上一帧同时进行，所以除第一帧外，之后的每一帧都可以隐藏设置输入的时间，从而提升性能。在异步模式下，每次返回的推理结果都是上一帧的。该参数的默认值为 False。 （目前版本该参数暂不支持）
	core_mask：设置运行时的 NPU 核心。支持的平台为 RK3588，支持的配置如下： RKNN.NPU_CORE_AUTO：表示自动调度模型，自动运行在当前空闲的 NPU 核上。 RKNN.NPU_CORE_0：表示运行在 NPU0 核心上。 RKNN.NPU_CORE_1：表示运行在 NPU1 核心上。RKNN.NPU_CORE_2：表示运行在 NPU2 核心上。RKNN.NPU_CORE_0_1：表示同时运行在 NPU0、NPU1 核心上。 RKNN.NPU_CORE_0_1_2：表示同时运行在 NPU0、NPU1、NPU2 核心上。 默认值为 RKNN.NPU_CORE_AUTO。
返回值	0：初始化运行时环境成功
	-1：初始化运行时环境失败

举例如下：

```
#初始化运行时环境
ret=rknn.init_runtime(target='rk3588')
if ret!=0:
    print('Initruntimeenvironmentfailed!')
exit(ret)
```

4.8 模型推理

在进行模型推理前，必须先构建或加载一个 RKNN 模型。

API	inference

描述	对当前模型进行推理，并返回推理结果。 如果 RKNN-Toolkit2 运行在 PC 上，且初始化运行环境时设置 target 为 RockchipNPU 设备，得到的是模型在硬件平台上的推理结果。 如果 RKNN-Toolkit2 运行在 PC 上，且初始化运行环境时没有设置 target，得到的是模型在模拟器上的推理结果。
参数	inputs：待推理的输入，如经过 cv2 处理的图片。格式是 ndarraylist。
	data_format：数据模式，可以填以下值：“nchw”,“nhwc”。默认值为“nhwc”。只对 4 维的输入有效。
	inputs_pass_through：将输入透传给 NPU 驱动。非透传模式下，在将输入传给 NPU 驱动之前，工具会对输入进行减均值、除方差等操作；而透传模式下，不会做这些操作。这个参数的值是一个数组，比如要透传 input0，不透传 input1，则这个参数的值为[1,0]。默认值为 None，即对所有输入都不透传。
返回值	results：推理结果，类型是 ndarraylist。

举例如下：

对于分类模型，如 mobilenet_v1，代码如下（完整代码参考 example/tflite/mobilent_v1）：

```
……
outputs=rknn.inference(inputs=[img])
show_outputs(outputs)……
```

输出的 TOP5 结果如下：

```
-----TOP5-----
[156]:0.93310546875
[155]:0.0555419921875
[205284]:0.003704071044921875
[205284]:0.003704071044921875
```

4.9 量化精度分析

该接口的功能是进行浮点、量化推理并产生每层的数据，用于量化精度分析。

API	accuracy_analysis
描述	推理并产生快照，也就是 dump 出每一层的 tensor 数据。会 dump 出包括 fp32 和 quant 两种数据类型的快照，用于计算量化误差。 注： 该接口只能在 build 或 hybrid_quantization_step2 之后调用，并且原始模型应该为非量化的模型，否则会调用失败。 该接口使用的量化方式与 config 中指定的一致。
参数	inputs：图像（jpg/png/bmp/npy 等）路径 list。 注：当 npy 文件维度为 4 时，需要将 layout 转为 nhwc。（其余维度不需做此转换）
	output_dir：输出目录，所有快照都保存在该目录（默认输出目录名为‘snapshot’）。如果没有设置 target，在 output_dir 下会输出： simulator 目录：保存整个量化模型在 simulator 上完整运行时每一层的结果（已转成 float32）； golden 目录：保存整个浮点模型在 simulator 上完整跑下来时每一层的结果; error_analysis.txt：记录在 simulator 上量化模型逐层运行时每一层的结果与 golden 浮点模型逐层运行时每一层的结果的余弦距离（entire_errorcosine），以及量化模型取上一层的浮点结果作为输入时，输出与浮点模型的余弦距离（per_layer_errorcosine） 如果有设置 target，则在 output_dir 里还会多输出： runtime 目录：保存整个量化模型在 NPU 上完整运行时每一层的结果（已转成 float32） error_analysis.txt：在上述记录的内容的基础上，还会记录量化模型在 simulator 上逐层运行时每一层的结果与 NPU 上逐层运行时每一层的结果的余弦距离（entire_errorcosine）
	target：目标硬件平台，支持“rk3566”、“rk3568”、“rk3588”、“rv1103”、“rv1106”，默认为 None。 如果设置了 target，则会获取 NPU 运行时每一层的结果，并进行精度的分析。
	device_id：设备编号，如果 PC 连接多台设备时，需要指定该参数，设备编号可以通过“list_devices”接口查看。默认值为 None。
返回值	0：成功
	-1：失败

4.10 混合量化

4.10.1 hybrid_quantization_step1

使用混合量化功能时，第一阶段调用的主要接口是 hybrid_quantization_step1，用于生成临时模型文件（{model_name}.model）、数据文件（{model_name}.data）和量化配置文件（{model_name}.quantization.cfg）。接口详情如下：

API	hybrid_quantization_step1
描述	根据加载的原始模型，生成对应的临时模型文件、配置文件和量化配置文件。
参数	dataset：见构建 RKNN 模型的 dataset 说明。
	rknn_batch_size：见构建 RKNN 模型的 rknn_batch_size 说明。
	proposal：产生混合量化的配置建议值。
	proposal_dataset_size：proposal 使用的 dataset 的张数。因为 proposal 功能比较耗时，所以默认只使用 1 张，也就是 dataset 里的第一张。
返回值	0：成功
	-1：失败

举例如下：

```
# 调用 hybrid_quantization_step1 产生量化配置文件
……
ret = rknn.hybrid_quantization_step1(dataset='./dataset.txt')
……
```

4.10.2 hybrid_quantization_step2

用于使用混合量化功能时生成 RKNN 模型，接口详情如下：

API	hybrid_quantization_step2
描述	接收临时模型文件、配置文件、量化配置文件和校正数据集作为输入，生成混合量化后的 RKNN 模型。
参数	model_input：第一步生成的临时模型文件，形如“{model_name}.model”。数据类型为字符串，必填参数。
	data_input：第一步生成的配置文件，形如“{model_name}.data”。数据类型为字符串，必填参数。
	model_quantization_cfg：经过修改后的模型量化配置文件，形如“{model_name}.quantization.cfg”。数据类型为字符串，必填参数。
返回值	0：成功
	-1：失败

举例如下：

```
# Call hybrid_quantization_step2 to generate hybrid quantized RKNN model
……
ret = rknn.hybrid_quantization_step2( model_input='./ssd_mobilenet_v2.model',
data_input='./ssd_mobilenet_v2.data',
model_quantization_cfg='./ssd_mobilenet_v2.quantization.cfg')
……
```